PRESTO-R

Predictive, Stability-Aware Reinforcement Learning Routing

Name: Yashwanth Gajula
Registration number: 24BDE0079
Slot: C2+TC2
Faculty: Dr. Navaneethan C

COMPUTER NETWORKS PROJECT REPORT

A simulation study of predictive routing policies that balance delivery, recovery behavior, and route stability.



Figure 1. PRESTO-R system overview, generated from the project implementation.

Evaluation scope: NSFNET and random geometric graph topologies; five traffic and failure scenarios; held-out test episodes.

Abstract

This project develops and evaluates PRESTO-R, a routing policy that combines predicted link-degradation risk with deep reinforcement learning and an explicit cost for changing routes. A discrete-time network simulator supplies noisy telemetry for one managed flow and background traffic. A gated recurrent unit (GRU) predictor estimates near-future link risk; a deep Q-network (DQN) selects among four candidate paths using path condition and risk features. Across held-out simulations, PRESTO-R reduced route changes substantially relative to predictive DQN while keeping packet delivery ratio (PDR) close to learned baselines. In NSFNET mixed episodes, it averaged 5.20 switches and 0.9625 PDR, compared with 20.99 switches and 0.9608 PDR for predictive DQN. The predictor onset F1 score was 0.5296, below the 0.70 project target, and PRESTO-R did not satisfy every predeclared comparison against reactive DQN. Results support a stability benefit within the tested simulator, while motivating improved prediction and validation on packet-level or real-network traces.

System Architecture and Design Methodology

The system separates simulation, risk estimation, route selection, and evaluation. The simulator produces noisy delay, loss, queue, utilization, and link-up observations while link health remains hidden. A predictor consumes a rolling telemetry window and estimates link risk for a selected horizon. The routing agent combines those estimates with current path measurements and selects one route from a fixed candidate set. A shared rollout and metrics pipeline evaluates heuristic and learned policies on matched seeds.

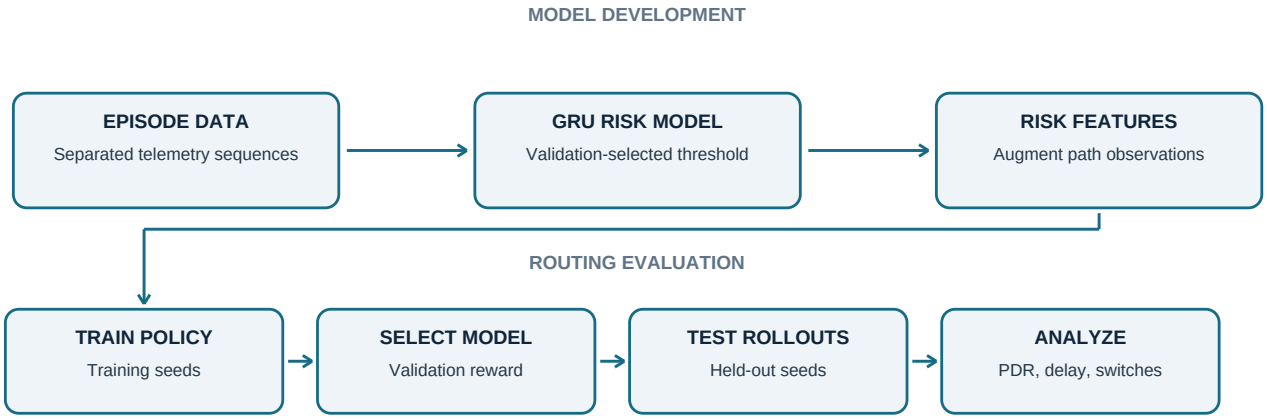


Figure 2. Data flow from simulated network through prediction and policy selection to evaluation.

Experimental design

Experiments use NSFNET and a random geometric graph (RGG), with steady, congestion, node-failure, fluctuation, and mixed scenarios. Each episode contains one managed source-destination flow and background traffic. Four loop-free candidate paths remain fixed for the episode. The default episode runs for 200 ticks after telemetry warm-up. Training, validation, and test seeds are separated; predictor thresholds and the switch penalty are selected on validation data, while held-out test episodes are reserved for final reporting.

Design element	Configuration
Network model	Discrete-time fluid simulator; 100 ms per tick
Topologies	NSFNET and random geometric graph
Scenarios	Steady, congestion, node failure, fluctuation, mixed
Routing choices	Four loop-free paths per managed flow
Telemetry / prediction	10-step history; default horizon H = 5
Evaluation	Held-out test seeds 10000-10199; paired seed comparisons

Implementation

The implementation is a reproducible pipeline: seeded simulation produces telemetry; telemetry datasets train risk models; policies use a shared environment interface; deterministic rollouts produce episode logs; and analysis code aggregates metrics, statistical comparisons, figures, and report outputs.

1. Network simulator and telemetry

The simulator models undirected links, background traffic, congestion, link fluctuations, and failure schedules in discrete time. Link health is latent, while the agent observes noisy delay, loss, queue, utilization, and availability. Candidate paths are generated by base delay and held fixed for each episode, keeping action meanings stable. A down link causes the selected path to deliver no traffic for that tick. This design supports large training and evaluation grids without claiming packet-level fidelity.

2. Link-risk prediction

The predictor package creates episode-separated sequence data and trains logistic regression and GRU models. The GRU uses a rolling telemetry window to estimate link degradation within a selected future horizon. Risk estimates augment route observations rather than replacing current measurements. Validation data supports threshold and horizon selection; the final test split remains held out. The recurrent sequence model is grounded in established gated recurrent network work [2].

3. Routing agents and reward

Three learned policies form the main comparison: reactive DQN uses current measurements, predictive DQN adds predicted risk, and PRESTO-R combines risk-aware observations with a route-switch penalty. DQN estimates action values for four candidate paths and learns from simulator interaction [1]. The reward balances delivered traffic and path quality with costs for delay, loss, and unnecessary route changes. The selected switch penalty is lambda = 0.5. A shortest-path failover heuristic provides a non-learning baseline. Candidate path generation follows the k-shortest loopless path approach [3].

4. Training and evaluation pipeline

Training uses 300,000 agent steps and multiple random seeds. NSFNET learned policies use five training seeds; RGG policies use three. Checkpoints are selected using validation reward. The evaluation grid spans both topologies, five scenarios, multiple prediction horizons, and robustness conditions including sudden failures, doubled measurement noise, and increased background load. Episode metrics include PDR, mean delay, recovery time where measurable, route switches, proactive rate, and unnecessary-switch rate. Statistical analysis uses matched held-out seeds with Holm correction for multiple comparisons.

5. Reproducibility and project modules

Component	Responsibility
presto/sim	Topology, traffic, failures, network dynamics
telemetry and predictor	Observation histories, datasets, risk models, evaluation
state, risk, reward, env	Agent state, risk interface, reward, environment API
policies and agents	Heuristic baselines and learned DQN policies
presto/eval	Rollouts, metrics, statistics, plots, report, success checks

Configuration files define simulator, topology, predictor, agent, and experiment-grid settings. Seed streams and resolved configurations are recorded with runs. The completed test grid contains 38,400 episode rows. The repository includes resumable grid evaluation, metric aggregation, statistical checks, figure generation, and report export. A model bundle and SHA-256 manifest support local integrity checks.

Findings and Result Analysis

The clearest measured effect is fewer route changes under the switch-penalized policy. On NSFNET mixed episodes, PRESTO-R reduced mean switches by 75.2% relative to predictive DQN while slightly increasing PDR. On RGG mixed episodes, PRESTO-R also used fewer switches, with PDR effectively comparable to predictive DQN. These are simulator results averaged over training-seed episode means, not guarantees for operational networks.

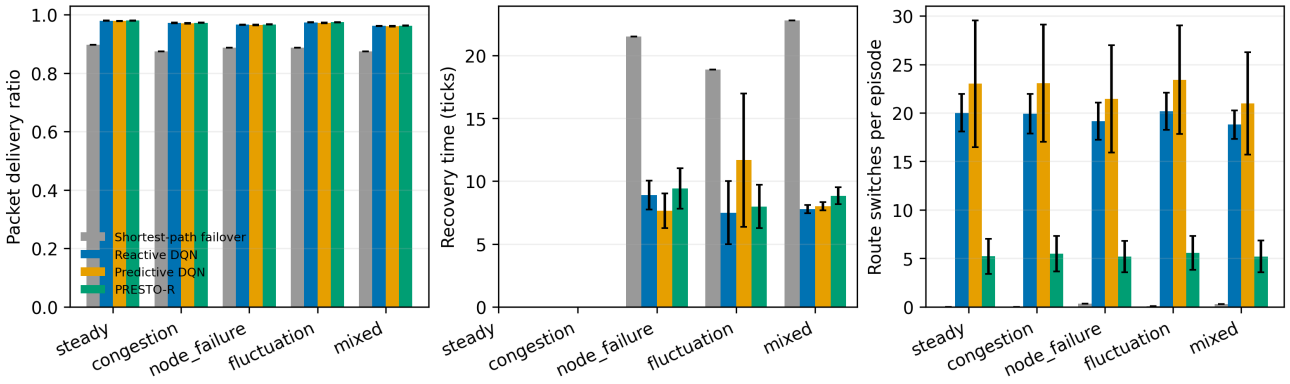


Figure 3. Held-out NSFNET results across the five evaluation scenarios.

Topology / scenario	Policy	PDR	Switches / episode
NSFNET mixed	Reactive DQN	0.9619 +/- 0.0008	18.799 +/- 1.458
NSFNET mixed	Predictive DQN	0.9608 +/- 0.0013	20.988 +/- 5.280
NSFNET mixed	PRESTO-R	0.9625 +/- 0.0005	5.197 +/- 1.628
RGG mixed	Predictive DQN	0.9546 +/- 0.0007	11.873 +/- 4.477
RGG mixed	PRESTO-R	0.9545 +/- 0.0010	1.913 +/- 0.264

Interpretation

On NSFNET mixed episodes, PRESTO-R achieved PDR 0.9625 versus 0.9608 for predictive DQN, alongside 5.197 versus 20.988 route changes per episode. On RGG, PDR was 0.9545 for PRESTO-R and 0.9546 for predictive DQN, while switches fell from 11.873 to 1.913. The lambda sweep shows a similar trade-off: larger penalties reduce switching, while PDR changes modestly.

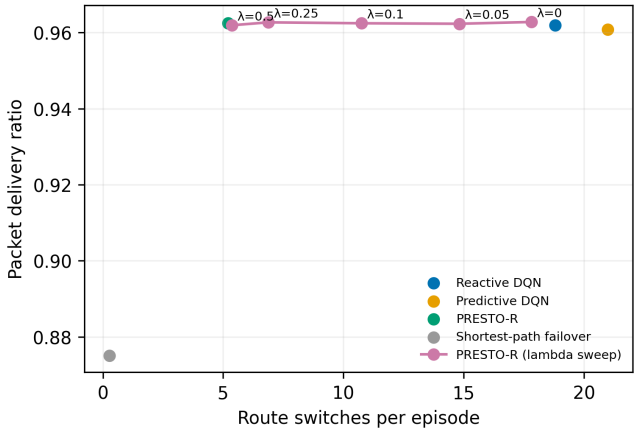


Figure 4. Mixed-scenario trade-off as the switch penalty changes.

Findings and Result Analysis

Predictor quality and criteria

The GRU achieved held-out all-sample F1 of 0.9769, but onset F1 was 0.5296. The high all-sample score should be interpreted cautiously because non-onset samples dominate the evaluation set. The onset score missed the predeclared target of 0.70, indicating that early warning remains a central weakness. Horizon sensitivity showed a trade-off: the H = 10 model had lower onset F1 than H = 3, while its routing outcomes retained similar PDR and fewer switches.

Predeclared check	Outcome	Evidence
PRESTO-R vs reactive, node failure	Not met	PDR 0.9667 vs 0.9662; recovery 9.440 vs 8.915
PRESTO-R vs reactive, fluctuation	Not met	PDR 0.9749 vs 0.9740; recovery 7.844 vs 7.386
PRESTO-R vs predictive, mixed	Met	5.197 vs 20.988 switches; PDR 0.9625 vs 0.9608
PRESTO-R vs predictive, fluctuation	Met	5.557 vs 23.419 switches; PDR 0.9749 vs 0.9726
Steady-state delay limit	Met	46.2766, below 50.9369 limit
Predictor onset F1 target	Not met	0.5296 against a target of 0.7000
Reproducibility	Met	Fresh rerun step and event hashes matched

Limitations

The environment is a fluid simulator, not a packet-level or production network. Experiments control one managed flow per episode, use synthetic failures, and generate predictor training telemetry under a specific failover policy. Sudden failures cannot be anticipated from precursor signals. The no-prediction plus switch-penalty factorial condition was not run, so the available ablation cannot isolate the independent contribution of prediction and switching cost. Some paired outcomes versus reactive DQN are inconclusive or do not meet predefined criteria. Optional NS-3 validation is not included.

Conclusion

PRESTO-R demonstrates a consistent reduction in route churn in the tested simulator while maintaining delivery ratios close to learned baselines. The evaluation does not establish universal performance gains: onset prediction missed its target, and not all comparisons against reactive DQN passed. Useful next steps are to improve onset detection, complete the missing factorial ablation, and validate against packet-level simulations or real network traces.

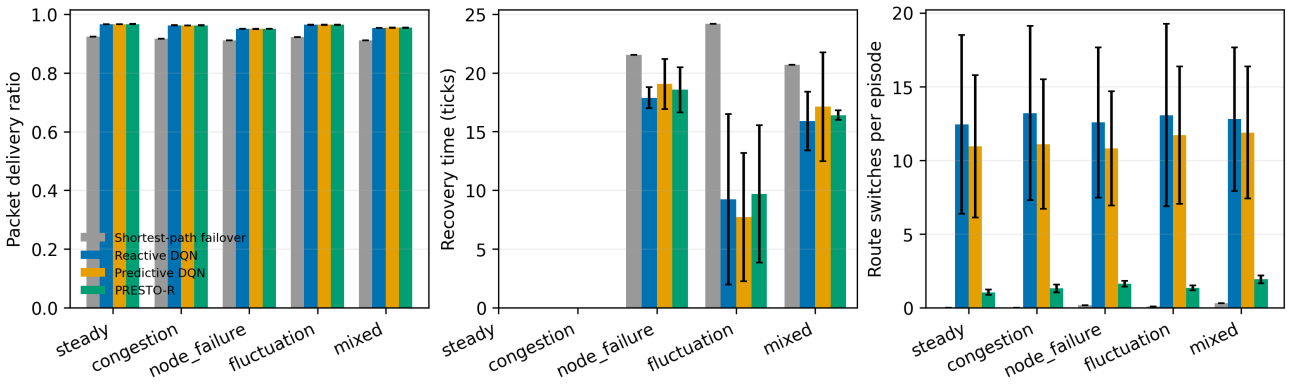


Figure 5. Held-out RGG results provide a second-topology check.

References

[1] V. Mnih et al., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, pp. 529-533, 2015. doi:10.1038/nature14236.

[2] K. Cho et al., "Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation," in *Proceedings of EMNLP*, 2014. doi:10.48550/arXiv.1406.1078.

[3] J. Y. Yen, "Finding the K Shortest Loopless Paths in a Network," *Management Science*, vol. 17, no. 11, pp. 712-716, 1971. doi:10.1287/mnsc.17.11.712.

[4] PRESTO-R project repository, source code, configurations, evaluation outputs, and generated report, Computer Networks project workspace, accessed October 2026.

[5] PRESTO-R project evaluation report and success-criteria summary, results/report.md and results/success_criteria.md, Computer Networks project workspace, accessed October 2026.

Project materials used

The project implementation, configuration files, saved evaluation tables, figures, and success-criteria record were used to prepare the system description and reported results. Metrics are reproduced as recorded in project outputs; interpretation is limited to the evaluated simulator settings.

PRESTO-R

Predictive, Stability-Aware Reinforcement Learning Routing